

ResumeForge — Gap Fix List

The uploaded spec's gaps, validated against the actual codebase

Prepared for Ojus

How to read this

The uploaded spec was reverse-engineered **from the public website only** — the author states plainly that the backend, code, and data flows were *not* inspected. So its gap analysis is a list of **hypotheses**, not findings. I checked each claim against your real code. Several of its top-priority “risks” turn out to be already solved; a few are real and serious.

Already handled — do NOT spend time here

The spec flags these as gaps or unknowns. They are already implemented:

- **Proof gating is server-side, not client-side.** The spec's #1 P0 risk is wrong. `/resume/export-check` blocks export on the server, `/evidence-resume` drops unbacked bullets server-side, and `_bullet_metric_supported()` is a deterministic regex guard that strips any metric not present in the cited evidence.
- **API keys are server-side only.** `GEMINI_API_KEY` lives in Render env vars; the frontend never touches it. No exposure.
- **Rate limiting exists.** A per-IP sliding window (default 30 requests / 60s) already guards every expensive AI endpoint via `RL_PATHS`.
- **Upload limits are enforced server-side** — 10 MB and 15 pages, in the backend, not just the browser.
- **Passwords are properly hashed** — PBKDF2-HMAC-SHA256, 200,000 iterations, per-user salt. (Argon2/bcrypt would be an upgrade, not a fix.)
- **Session expiry exists** — 30 days, enforced in `get_user_from_token`.
- **Cover letter generation exists** — the spec wrongly lists it as out of scope.
- **Error reporting + analytics exist** — Sentry (optional via `SENTRY_DSN`) and a privacy-safe events table.
- **Proof pages never expose source** — only user-approved items are rendered; READMEs are never dumped.
- **Non-developer path, Interview Prep, and the live voice Mock Interview are all built.**

P0 — Fix first (security & trust)

- **Extension session auto-connect (the spec is right — this is real).** The content script runs on `<all_urls>` and reads `rf_token` from any page's `localStorage`. Restrict the site-sync to the ResumeForge origin via an explicit allowlist, and scope/expire the token the extension receives.
- **Prompt injection via untrusted text.** READMEs, job descriptions, LinkedIn text, and pasted CVs go straight into Gemini prompts with no isolation. A malicious JD or README could manipulate a verdict or a tailored resume. Delimit and neutralise untrusted input, and validate model output (you already do this for metrics — extend the pattern).

- **Session token in localStorage.** Any XSS on the site hands over an account. Consider an httpOnly cookie, and tighten the XSS surface (you inject a lot of HTML via `innerHTML`).

P1 — Account lifecycle & brand invariants

- **Password reset / forgot-password flow — missing entirely.** Users who forget a password are permanently locked out.
- **Email verification on signup — missing.** Nothing stops fake accounts.
- **Full account deletion — missing.** You can delete imported GitHub evidence, but not the account and its data.
- **Define “Verified vs Inferred” deterministically.** Right now it’s a model judgement. Your entire brand promise rests on it, so write the rules: are forks excluded? tutorial code? commits authored by others in a shared repo?
- **Canvas editor contradicts the ATS promise.** Free-form, rotated, layered designs will not parse in an ATS while the headline says “ATS-friendly.” Add an explicit warning on canvas export (or an ATS check).
- **Shared proof pages can’t be revoked or expired.** `/p/{slug}` links are permanent and unlisted. Add revocation and an optional expiry.

P2 — Cost, abuse & reliability

- **No per-user quotas; anonymous users can trigger generation.** Rate limiting exists but is per-IP and in-memory (resets on restart, per-process). Add per-user quotas, auth-gate the expensive endpoints, and move the limiter to Redis if you ever run more than one instance.
- **No pricing or fair-use policy.** “Free” plus heavy LLM usage is not sustainable — this is a business gap as much as a technical one.
- **No telemetry when auto-fill breaks.** You’re more resilient than the spec assumed (ATS hints, then rules, then a cached form-map, then an AI fallback), but you have no signal when fills silently fail. Add fill-success telemetry.
- **No ATS validation of exported PDFs.** Nothing tests the output against a real ATS parser.
- **Analytics exist but aren’t surfaced.** Build the `/stats` page you already planned.

P3 — Later, but worth listing

- **Recruiter-page analytics — an easy win.** You *already* count views in `shared_proofs.views` on every `/p/{slug}` open. Just surface it: “3 recruiters opened your proof.”
- **No data export (JSON) of vault or resumes — portability.**
- **Accessibility.** Drag-drop and canvas editors are keyboard-inaccessible; no a11y statement.
- **Internationalisation / locale — English-only; A4 vs Letter, phone and date formats unspecified.**
- **Concurrency.** Editing the same resume in two tabs, or regenerating over an edited resume, is undefined.
- **Data retention and region not documented** (GDPR / India DPDP).
- **OAuth login (Google/GitHub) for main auth — currently email/password only; GitHub OAuth exists but only for Vault import, is env-gated, and is untested end-to-end.**

Answers to the spec’s “Open Questions”

- **Where is export gating enforced?** Server-side (`/resume/export-check`).
- **What powers the AI?** Google Gemini (`gemini-2.5-flash`); keys are strictly server-side.
- **Are proof pages unlisted or access-controlled?** Unlisted random-slug URLs with `noindex`,

no expiry, no revocation — see P1.

- **Is the extension on the Chrome Web Store?** Not yet — it's distributed as a ZIP.
- **Storage / retention / deletion?** Supabase Postgres in prod, SQLite locally. No documented retention policy; only GitHub-evidence deletion exists.
- **Monetisation?** None yet — see P2.

Bottom line: the spec's two P0s were half-wrong (proof gating and key exposure are already server-side), but it correctly caught the extension session risk. The genuinely urgent work is: extension origin allowlist, prompt-injection isolation, password reset, and a real quota/pricing model.